

services\data\api_client.py

```
import requests
import json
import time
from datetime import datetime, timezone, timedelta
from typing import List, Dict, Any, Optional
import config
import threading
import logging

# Set up logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class NVDApiClient:
    """Enhanced NVD API client with database support and aggressive memory optimization"""

    def __init__(self):
        self.base_url = config.NVD_API_URL
        self.api_key = config.NVD_API_KEY
        self.timeout = config.API_TIMEOUT
        self.last_request_time = 0

    # Proper rate limiting based on API key availability
    if self.api_key:
        self.min_request_interval = 30 / 50 # 50 requests per 30 seconds with key
    else:
        self.min_request_interval = 30 / 5 # 5 requests per 30 seconds without key

    # Session for connection reuse
    self.session = requests.Session()
    if self.api_key:
        self.session.headers.update({"apiKey": self.api_key})

    # Lightweight cache for metadata only
    self._cache_timestamps = {}
    self._cache_lock = threading.Lock()

    # Database integration
    from database import db_manager
    self.db = db_manager

    def _rate_limit(self):
        """Proper rate limiting with logging"""
        elapsed = time.time() - self.last_request_time
        if elapsed < self.min_request_interval:
            sleep_time = self.min_request_interval - elapsed
            time.sleep(sleep_time)
            self.last_request_time = time.time()

    def _is_cache_valid(self, cache_key: str, max_age: int = 900) -> bool:
        """Check if cache entry is still valid (default 15 minutes)"""
        with self._cache_lock:
            if cache_key not in self._cache_timestamps:
                return False
            age = time.time() - self._cache_timestamps[cache_key]
            return age < max_age

    def _set_cache_timestamp(self, cache_key: str):
        """Set cache timestamp"""
        with self._cache_lock:
            self._cache_timestamps[cache_key] = time.time()

    def get_cves_last_30_days(self) -> List[Dict[str, Any]]:
        """Get CVEs from last 30 days - OPTIMIZED for memory"""
        cache_key = "cves_30_days"

    # Try database first if available
    if self.db.use_database:
        # Check if we have recent data in database
        metadata = self.db.get_cache_metadata(cache_key)
        if metadata:
            last_updated = metadata.get('last_updated')
```

```

if last_updated:
age = datetime.now(timezone.utc) - last_updated.replace(tzinfo=timezone.utc)
if age.total_seconds() < 900: # 15 minutes
logger.info("Using database cached CVE data")
cves = self.db.get_all_cves()
if cves:
return cves

# Check memory cache timestamp
if self._is_cache_valid(cache_key, max_age=900):
logger.info("Database cache still valid, fetching from DB")
if self.db.use_database:
cves = self.db.get_all_cves()
if cves:
return cves

logger.info("Fetching fresh CVE data from API")

# Calculate date range
end_date = datetime.now(timezone.utc)
start_date = end_date - timedelta(days=30)

all_cves = []
start_index = 0
results_per_page = 2000
max_results = 10000

try:
while start_index < max_results:
params = {
"resultsPerPage": results_per_page,
"startIndex": start_index,
"pubStartDate": start_date.strftime("%Y-%m-%dT%H:%M:%S.000"),
"pubEndDate": end_date.strftime("%Y-%m-%dT%H:%M:%S.000")
}

logger.info(f"API request: startIndex={start_index}, resultsPerPage={results_per_page}")

self._rate_limit()
response = self.session.get(
self.base_url,
params=params,
timeout=self.timeout
)

if response.status_code != 200:
logger.error(f"API request failed: {response.status_code} - {response.text}")
break

data = response.json()
vulnerabilities = data.get("vulnerabilities", [])
total_results = data.get("totalResults", 0)

logger.info(f"Got {len(vulnerabilities)} CVEs (total available: {total_results})")

if not vulnerabilities:
break

# Process CVEs in SMALLER batches to reduce memory
batch = []
for item in vulnerabilities:
cve_data = item.get("cve", {})
processed = self._process_cve_item(cve_data)
if processed:
batch.append(processed)

# Save in smaller batches of 500
if len(batch) >= 500 and self.db.use_database:
self.db.save_cves_batch(batch)
all_cves.extend(batch)
batch = [] # Clear batch immediately
del batch
batch = []

# Save remaining items
if batch:
if self.db.use_database:

```

```

self.db.save_cves_batch(batch)
all_cves.extend(batch)
del batch

# Check if we've got all results or hit our limit
if len(vulnerabilities) < results_per_page or start_index + len(vulnerabilities) >=
total_results:
break

start_index += len(vulnerabilities)

# Safety limit
if start_index >= max_results:
logger.warning(f"Hit safety limit of {max_results} results")
break

# Sort by published date (newest first)
all_cves.sort(key=lambda x: x.get('Published', ''), reverse=True)

# Update cache metadata
if self.db.use_database:
self.db.update_cache_metadata(cache_key, len(all_cves))
self._set_cache_timestamp(cache_key)

logger.info(f"Successfully fetched and cached {len(all_cves)} CVEs")
return all_cves

except requests.exceptions.RequestException as e:
logger.error(f"API request exception: {e}")
# Try database as fallback
if self.db.use_database:
logger.info("Trying database as fallback")
cves = self.db.get_all_cves()
if cves:
return cves
return []
except Exception as e:
logger.error(f"Unexpected error: {e}")
# Try database as fallback
if self.db.use_database:
cves = self.db.get_all_cves()
if cves:
return cves
return []

def _process_cve_item(self, cve_data: Dict) -> Optional[Dict]:
"""Convert NVD CVE format to our internal format"""
try:
cve_id = cve_data.get("id", "")
if not cve_id:
return None

# Extract description
description = ""
for desc in cve_data.get("descriptions", []):
if desc.get("lang") == "en":
description = desc.get("value", "")
break

# Extract severity and CVSS
severity = "UNKNOWN"
cvss_score = None
metrics = cve_data.get("metrics", {})

# Try CVSS v4.0, v3.1, v3.0, then v2.0
for version in ["cvssMetricV40", "cvssMetricV31", "cvssMetricV30", "cvssMetricV2"]:
if version in metrics and metrics[version]:
try:
metric = metrics[version][0]
cvss_data = metric.get("cvssData", {})
potential_severity = cvss_data.get("baseSeverity", "")
potential_score = cvss_data.get("baseScore", None)

if potential_severity and potential_severity.upper() != "UNKNOWN":
severity = potential_severity
cvss_score = potential_score
break

```

```

except (IndexError, KeyError):
    continue

# Extract CWE
cwe = None
for weakness in cve_data.get("weaknesses", []):
    for desc in weakness.get("description", []):
        if desc.get("lang") == "en":
            cwe = desc.get("value")
            break
    if cwe:
        break

# Extract references
references = []
for ref in cve_data.get("references", []):
    url = ref.get("url")
    if url:
        references.append(url)

return {
    "ID": cve_id,
    "Description": description or "No description available",
    "Severity": severity.upper() if severity else "UNKNOWN",
    "CVSS_Score": cvss_score,
    "CWE": cwe,
    "Published": cve_data.get("published", ""),
    "lastModified": cve_data.get("lastModified", ""),
    "References": references,
    "Products": [],
    "metrics": metrics # Preserving full metrics
}

except Exception as e:
    print(f"[API] Error processing CVE: {e}")
    return None

def get_cve_detail(self, cve_id: str) -> Dict[str, Any]:
    try:
        # 1. Check database first
        if self.db.use_database:
            db_cve = self.db.get_cve_detail(cve_id)
            if db_cve:
                logger.info(f"CVE {cve_id} retrieved from database")
                return db_cve

        # 2. Check local cache of recent CVEs
        local_cves = self.get_cves_last_30_days()
        matching_cve = next((cve for cve in local_cves if cve['ID'] == cve_id), None)

        if matching_cve:
            logger.info(f"CVE {cve_id} retrieved from local cache")
            return matching_cve

        # 3. Fetch from NVD API with comprehensive error handling
        logger.info(f"Attempting to fetch CVE {cve_id} from NVD API")
        self._rate_limit()

        response = self.session.get(
            f"{self.base_url}?cveId={cve_id}",
            timeout=self.timeout
        )

        # Detailed logging for API response
        logger.info(f"API Response Status: {response.status_code}")
        logger.info(f"API Response Text: {response.text[:500]}..." ) # Limit log length

        if response.status_code != 200:
            logger.warning(f"API request failed for {cve_id}: {response.status_code}")
            return self._generate_fallback_cve(cve_id, f"API request failed with status {response.status_code}")

        try:
            data = response.json()
        except json.JSONDecodeError:
            logger.error(f"Failed to parse JSON for {cve_id}")
            return self._generate_fallback_cve(cve_id, "Invalid API response format")

```

```

vulnerabilities = data.get("vulnerabilities", [])
if not vulnerabilities:
    logger.warning(f"No vulnerability data found for {cve_id}")
    return self._generate_fallback_cve(cve_id, "No vulnerability data available")

# Process the first vulnerability (assuming single CVE)
cve_data = vulnerabilities[0].get("cve", {})
processed_cve = self._process_cve_item(cve_data)

if not processed_cve:
    logger.warning(f"Unable to process CVE details for {cve_id}")
    return self._generate_fallback_cve(cve_id, "Unable to process CVE details")

# Optionally save to database
if self.db.use_database:
    self.db.save_cve_detail(processed_cve)

return processed_cve

except Exception as e:
    logger.error(f"Unexpected error retrieving CVE {cve_id}: {e}")
    return self._generate_fallback_cve(cve_id, str(e))

def _generate_fallback_cve(self, cve_id: str, error_message: str = "") -> Dict[str, Any]:
    """Generate a fallback CVE structure when no details are available"""
    return {
        'ID': cve_id,
        'Description': f'No details available for {cve_id}. {error_message}',
        'Severity': 'UNKNOWN',
        'CVSS_Score': None,
        'CWE': 'Unknown',
        'Published': '',
        'lastModified': '',
        'References': [],
        'Products': [],
        'metrics': {},
        'Error': error_message
    }

def clear_cache(self):
    """Clear all cached data"""
    with self._cache_lock:
        self._cache_timestamps.clear()

# Clear database cache if available
if self.db.use_database:
    self.db.clear_all_cves()

logger.info("Cache cleared")

def get_cache_stats(self) -> Dict[str, Any]:
    """Get cache statistics"""
    with self._cache_lock:
        stats = {
            'cached_entries': len(self._cache_timestamps),
            'cache_ages': {}
        }

    current_time = time.time()
    for key, timestamp in self._cache_timestamps.items():
        age_minutes = int((current_time - timestamp) / 60)
        stats['cache_ages'][key] = f"{age_minutes} minutes"

# Add database stats if available
if self.db.use_database:
    stats['database'] = self.db.get_stats()

return stats

# Global API client instance
api_client = NVDApiClient()

```